

Fördjupning i Polars – Muntlig presentation

1. Jag har fördjupat mig i **Polars**. Det är ett bibliotek för att jobba med tabelldata (DataFrame), ungefär som **Pandas**. Jag ville ta reda på om Polars verkligen är snabbare än Pandas, och varför.

2. I kursen har vi mest kört Pandas. Men Polars dyker upp hela tiden i artiklar och diskussioner. Så i stället för att bara tro på det ville jag testa själv. Jag hade tre mål. Först lära mig grunderna: läsa in, filtrera, sortera och gruppera data. Sedan förstå varför Polars brukar vara snabbare. Och till sist göra en egen jämförelse på en större datamängd. Det här är relevant för en Data Scientist, eftersom data kan bli väldigt stor. till exempel mätvärden från elnätet. Det känner jag till från mitt tidigare jobb.

3. Det finns fyra begrepp som förklarar mycket.

1. **Kolumnbaserad lagring**. Polars bygger på Apache Arrow och lagrar data kolumn för kolumn. Då går det snabbt att räkna på hela kolumner, till exempel en summa eller ett medelvärde.

2. **Multi-threading**. Polars använder flera CPU-kärnor samtidigt, och jag behöver inte skriva någon extra kod för det. Pandas kör för det mesta på en kärna.

3. **Lazy evaluation**. Jag kan bygga upp en hel kedja av steg först. Polars optimerar planen innan något körs. Det liknar hur en SQL-databas planerar en fråga.

4. **Striktare typning**. Polars är noggrannare med datatyper. Det kan kännas ovanligt, men det ger färre överraskningar.

4. Jag byggde en egen datamängd med **5 475 rader**. Den visar daglig elförbrukning för fem städer under 2025, uppdelat på hushåll, industri och kommersiell verksamhet. Jag lade in högre förbrukning på vintern. Jag lade också in några saknade värden i temperaturen, så att jag kunde öva på det. Datan är alltså syntetisk.

Jag jobbade i en Jupyter-notebook i sex steg. Läs in med **read_csv**. Filtrera och sortera. Skapa nya kolumner med **with_columns**. Gruppera med **group_by** och **agg**. Testa lazy med **scan_csv** och **collect**. Och till sist jämföra mot Pandas.

5. Här ser ni samma uppgift i båda biblioteken: total förbrukning per region, sorterat från högst till lägst. Uppe är Polars. Jag börjar med **group_by** på region. Sedan kommer **agg**, där jag säger vilken kolumn som ska summeras och vad resultatet ska heta. Till sist **sort**. Nere är Pandas, och det blir samma resultat. Tabellen visar svaret. Stockholm använder mest, 470 684 kWh, och

Umeå minst. Det jag gillar är att allt står i samma uttryck i Polars. Det svåraste i början var att tänka i uttryck med **pl.col()**, i stället för att behandla kolumner som vanliga variabler.

6. Min egen datamängd är för liten för att man ska se någon skillnad. Så jag skapade en testfil med **en miljon rader**. Jag mätte tiden för att läsa in filen, filtrera, och sedan gruppera och räkna medelvärde. Pandas tog 1,2 sekunder. Polars i vanligt läge tog 0,52 sekunder, ungefär **2,3 gånger snabbare**. Polars i lazy-läge tog 0,46 sekunder, ungefär **2,6 gånger snabbare**. Men, och det här är min viktigaste begränsning: jag körde varje test bara **en gång**, på min egen dator. Så det är inget exakt benchmark. Det visar en tendens, inte ett facit.

7. Sammanfattat: datan är syntetisk och jag höll mig till grunderna. Ett bättre test hade kört flera gånger och jämfört median. För en Data Scientist tror jag att det är bra att kunna **både Pandas och Polars**, och välja efter situation. Särskilt när man jobbar med stora mätdata, som från elnätet.

Som nästa steg vill jag testa en riktig öppen datamängd, till exempel från Energimyndigheten, och prova streaming för filer som inte får plats i minnet.

Tack! Har ni några frågor?